

Design & Analysis of Algorithms

Lab 1: Algorithm Performance Analysis Workbench

Name: Anupam Kumar Singh

Roll No: 2501730218

Email: 2501730218@krmu.edu.in

Q1. 1.2 Linear Search and Binary Search Comparison (5 marks)

```
def compare_search_algorithms(arr, target):
    linear_index = -1
    linear_comparison = 0

    for i in range(len(arr)):
        linear_comparison+=1
        if arr[i] == target:
            linear_index = i
            break

    binary_index = -1
    binary_comparsion = 0
    left = 0
    right = len(arr) - 1

    while left <= right :
        mid = (left+right) //2
        binary_comparsion+=1

        if arr[mid] == target:
            binary_index = mid
            right = mid - 1

        elif arr[mid] < target :
            left = mid + 1

    else:
        right = mid - 1

    if linear_comparison < binary_comparsion :
        better = "Linear Search"
    elif binary_comparsion < linear_comparison:
        better = "Binary Search"
    else:
        better = "Both Equal"

    return["Search Comparison Report",
           "Linear Search",
           f"Index: {linear_index}",
           f"Comparisons: {linear_comparison}",
           "Binary Search",
           f"Index: {binary_index}",
           f"Comparisons: {binary_comparsion}",
           f"Better Algorithm: {better}"
          ]
```

Q2. 1.3 Bubble Sort and Insertion Sort Performance Comparison (10 marks)

```
def compare_bubble_insertion(random_data, sorted_data, reverse_data):
    def bubble_sort(data):
        arr = data[:]
        comparisons = 0
        swaps = 0
        n = len(arr)

        for i in range(n-1):
            swapped = False
            for j in range(n-1-i):
                comparisons+=1
                if arr[j] > arr[j+1]:
                    arr[j],arr[j+1] = arr[j+1] , arr[j]
                    swaps+=1
                    swapped = True
```

```

        if not swapped:
            break

    return arr, comparisons, swaps

def insertion_sort(data):
    arr = data[:]
    comparisons = 0
    shifts = 0
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0:
            comparisons += 1
            if arr[j] > key:
                arr[j + 1] = arr[j]
                shifts += 1
                j -= 1
            else:
                break
        arr[j + 1] = key

    return arr, comparisons, shifts

datasets = [("Random Dataset", random_data),
            ("Sorted Dataset", sorted_data),
            ("Reverse Dataset", reverse_data)]
result = ["Sorting Performance Report"]

for name, data in datasets:
    bubble_result, bubble_comp, bubble_swaps = bubble_sort(data)
    insertion_result, insertion_comp, insertion_shifts = insertion_sort(data)

    if bubble_comp < insertion_comp:
        better = "Bubble Sort"
    elif insertion_comp < bubble_comp:
        better = "Insertion Sort"
    else:
        better = "Both Equal"

    result.append(name)

    result.append("Bubble Sorted: " + " ".join(map(str, bubble_result)))
    result.append(f"Bubble Comparisons: {bubble_comp}")
    result.append(f"Bubble Swaps: {bubble_swaps}")
    result.append("Insertion Sorted: " + " ".join(map(str, insertion_result)))
    result.append(f"Insertion Comparisons: {insertion_comp}")
    result.append(f"Insertion Shifts: {insertion_shifts}")
    result.append(f"Better Algorithm: {better}")

return result

```